



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria en
Ingeniería y Tecnologías Avanzadas



Diseño de propuesta de fragmentación para Ejercicio 3

Alumnos: González Vázquez Rodrigo
Mendoza Xicohtencatl Héctor Jair

Profesor: De la Cruz Sosa Carlos

Materia: Base de datos distribuidas

Grupo: 3TM2

Fecha: 24 de septiembre de 2026

Índice

Planteamiento.....	3
Consulta exploratoria (estado real de los datos).....	3
Grafo relacional del ejercicio	4
Fragmentación horizontal primaria de <i>Customer</i>	5
Definición de las regiones (atributo Group)	5
Análisis de conteo de filas: subdivisión de Norteamérica	5
Conjunto de predicados simples Pr	6
Algoritmo COM_MIN para obtener a Pr'	6
Conjunto de minitérminos M.....	7
Fragmentos primarios resultantes.....	8
Fragmento n+1 y regla de completitud	8
Fragmentación derivada	9
Validación de las tres reglas de correctitud.....	9
Completitud	9
Reconstrucción.....	9
Disyunción (fragmentos ajenos)	9
Conclusión.....	10

Planteamiento

El Ejercicio 3 pide distribuir la cartera de clientes de **AdventureWorks2022** en nodos geográficos, de modo que cada región administre y consulte de forma autónoma los datos de sus propios compradores. La relación global para fragmentar es *Sales.Customer*.

En lugar de fragmentar por un atributo secuencial como *CustomerID* (que solo refleja el orden de registro), elegimos el atributo *TerritoryID*, porque agrupar a los clientes por su ubicación geográfica es lo que tiene sentido para la lógica del negocio y para las consultas locales por región.

Consulta exploratoria (estado real de los datos)

Antes de definir los fragmentos, era necesario saber cómo estaban distribuidos los datos. La base cuenta con más de 19,800 clientes. Se ejecutó la siguiente consulta para conocer el volumen real por territorio:

```
SELECT Sales.SalesTerritory.TerritoryID,
       Sales.SalesTerritory.Name,
       Sales.SalesTerritory.[Group],
       COUNT(Sales.Customer.CustomerID) AS TotalClientes
FROM Sales.SalesTerritory
JOIN Sales.Customer
      ON Sales.SalesTerritory.TerritoryID = Sales.Customer.TerritoryID
GROUP BY Sales.SalesTerritory.TerritoryID,
         Sales.SalesTerritory.Name,
         Sales.SalesTerritory.[Group]
ORDER BY TotalClientes DESC;
```

Resultado obtenido (ordenado por volumen):

TerritoryID	Name	Group	TotalClientes
4	Southwest	North America	4,696
9	Australia	Pacific	3,665
1	Northwest	North America	3,520
10	United Kingdom	Europe	1,991
7	France	Europe	1,884
8	Germany	Europe	1,852
6	Canada	North America	1,791
5	Southeast	North America	176
3	Central	North America	132
2	Northeast	North America	113
Total			19,820

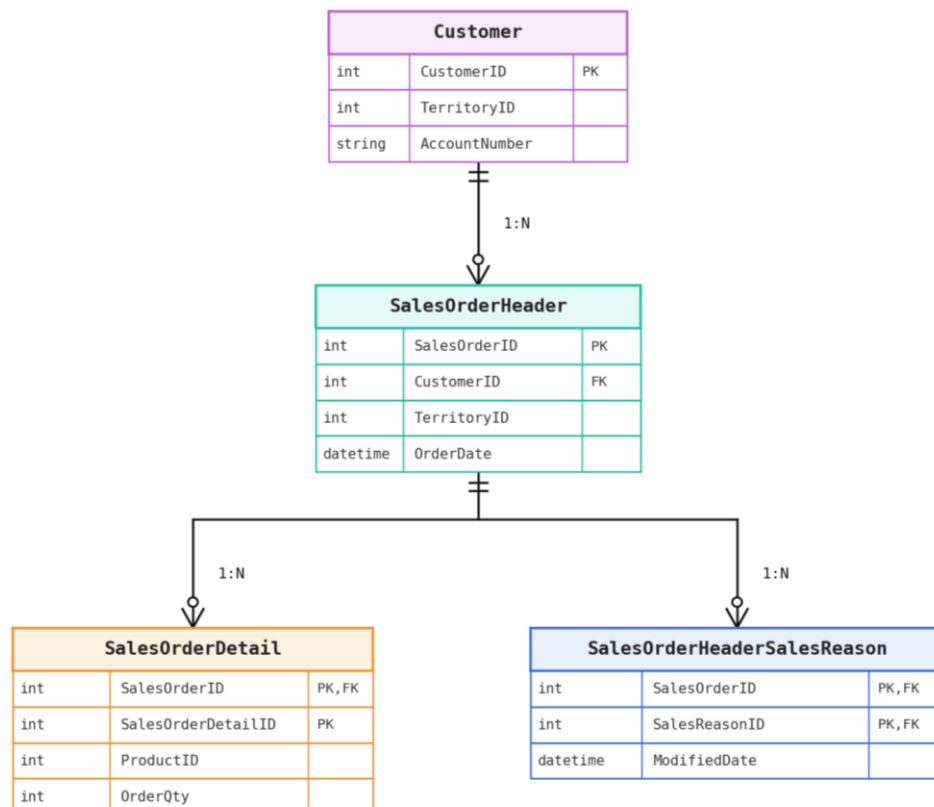
Grafo relacional del ejercicio

El ejercicio no involucra solo a los clientes dado que existen relaciones jerárquicas que dependen de *Sales.Customer*. La tabla **propietaria** de nuestro diseño es *Sales.Customer*, y de ella se derivan las siguientes tablas **miembro**:

- ***Sales.SalesOrderHeader*** : Cada orden de venta pertenece a un cliente (*CustomerID*). Es miembro de *Customer* y, a su vez, propietaria de las dos tablas.
- ***Sales.SalesOrderDetail***: Es miembro de ***SalesOrderHeader*** por ***SalesOrderID***.
- ***Sales.SalesOrderHeaderSalesReason***: Tabla de cruce que asocia cada orden con sus motivos de compra. También es miembro de ***SalesOrderHeader*** por ***SalesOrderID*** (es la tercera tabla que se fragmenta de forma derivada, hermana de ***SalesOrderDetail***).

Esta estructura corresponde al patrón de Özsu en el que una relación (*SalesOrderHeader*) es al mismo tiempo miembro de una y propietaria de otras, formando una **cadena de fragmentación derivada**:

Customer → *SalesOrderHeader* → {*SalesOrderDetail*, *SalesOrderHeaderSalesReason*}.



SalesOrderHeader posee su propia columna *TerritoryID*, por lo que podría fragmentarse de forma primaria. Elegimos derivarla desde *Customer* por *CustomerID* a propósito, para que todas las órdenes de un cliente residan en el mismo nodo que el cliente.

Fragmentación horizontal primaria de *Customer*

COM_MIN necesita conocer las aplicaciones, porque la minimalidad se define en términos de relevancia (Regla 1): un predicado es relevante solo si al menos una aplicación accede de manera distinta a los fragmentos que ese predicado genera. Para este ejercicio consideramos dos aplicaciones:

- **A1 (autonomía regional):** cada nodo consulta y administra únicamente a los clientes de su propia región geográfica (Norteamérica, Europa, Pacífico).
- **A2 (balanceo de carga):** ningún nodo debe concentrar un volumen desproporcionado de clientes respecto de los demás, para evitar cuellos de botella.

Definición de las regiones (atributo *Group*)

La aplicación A1 define la partición natural por el atributo *Group* de ***Sales.SalesTerritory***. Agrupando los 10 territorios por su *Group* obtenemos exactamente 3 regiones:

Región (Group)	Territorios incluidos	Clientes
North America	1, 2, 3, 4, 5, 6	10,428
Europe	7, 8, 10	5,727
Pacific	9	3,665

Análisis de conteo de filas: subdivisión de Norteamérica

Si nos quedáramos con las 3 regiones tal cual, Norteamérica (10,428) tendría casi el doble de Europa (5,727) y casi el triple del Pacífico (3,665) lo cual viola la aplicación A2 (balanceo). Por eso aplicamos un análisis de conteo de filas para subdividir **únicamente** la región de Norteamérica, dejando Europa y Pacífico intactas.

Buscando subregiones de tamaño parecidos (5,200 clientes) donde la partición de Norteamérica queda:

Subregión de Norteamérica	Territorios	Clientes
NA1	4 (SW), 5 (SE), 3 (Central), 2 (NE)	5,117
NA2	1 (NW), 6 (Canadá)	5,311

Resultado del balanceo final por nodo de datos:

$$NA1 = 5,117,$$

$$NA2 = 5,311,$$

$$Europa = 5,727$$

$$Pacífico = 3,665.$$

Las cuatro quedan del mismo orden de magnitud, cumpliendo A2 sin romper A1.

Conjunto de predicados simples Pr

De las regiones y subdivisión por conteo de NA obtenemos el conjunto inicial de predicados simples Pr sobre $TerritoryID$:

$$Pr = \{ p1, p2, p3, p4, p5 \}$$

$$p1 : TerritoryID \textbf{IN} (1,2,3,4,5,6) \quad -- \text{Norteamérica (Group)}$$

$$p2 : TerritoryID \textbf{IN} (7,8,10) \quad -- \text{Europa (Group)}$$

$$p3 : TerritoryID = 9 \quad -- \text{Pacífico (Group)}$$

$$p4 : TerritoryID \textbf{IN} (2,3,4,5) \quad -- \text{Subregión NA-1 (conteo)}$$

$$p5 : TerritoryID \textbf{IN} (1,6) \quad -- \text{Subregión NA-2 (conteo)}$$

Algoritmo COM_MIN para obtener a Pr'

COM_MIN toma Pr y produce un conjunto **completo y mínimo** Pr' .

Aplicamos sus reglas:

1. **Complejidad:** Toda tupla debe tener la misma probabilidad de acceso dentro del fragmento al que cae $p2, p3, p4$ y $p5$ cada cliente queda asignado a una única región/subregión que una aplicación consulta de manera homogénea, por lo que el conjunto es completo.
2. **Minimalidad:** Un predicado se conserva solo si alguna aplicación accede distinto a los fragmentos que produce. Observamos que $p1 \equiv p4 \vee p5$: $p1$ no genera ninguna partición de acceso que $p4$ y $p5$ no generen ya. Como ninguna aplicación accede a toda Norteamérica como un bloque (A1 trabaja por subregión y A2 exige separarlas), $p1$ es *no relevante* y se elimina.

3. **Verificación:** Los cuatro predicados restantes son relevantes (cada uno aísla un conjunto de filas que alguna aplicación consulta por separado) y no se puede quitar ninguno sin perder una partición útil.

El conjunto resultante es:

$$Pr' = \{ p2, p3, p4, p5 \}$$

Conjunto de minitérminos M

Un minitérmino tiene la forma general:

$$m_i = (p2 *) \wedge (p3 *) \wedge (p4 *) \wedge (p5 *)$$

Como ***TerritoryID*** toma un solo valor por fila, los conjuntos (7,8,10), (9), (2,3,4,5) y (1,6) son disjuntos entre sí. Por lo tanto, cualquier minitérmino que afirme dos predicados positivos a la vez (p. ej. $p2 \wedge p4$) es contradictorio y se elimina. Sobreviven únicamente los minitérminos con un solo predicado positivo, más el minitérmino con todo negado:

$$m1 = p4 \wedge \neg p2 \wedge \neg p3 \wedge \neg p5 \equiv TerritoryID \text{ IN } (2,3,4,5)$$

$$m2 = p5 \wedge \neg p2 \wedge \neg p3 \wedge \neg p4 \equiv TerritoryID \text{ IN } (1,6)$$

$$m3 = p2 \wedge \neg p3 \wedge \neg p4 \wedge \neg p5 \equiv TerritoryID \text{ IN } (7,8,10)$$

$$m4 = p3 \wedge \neg p2 \wedge \neg p4 \wedge \neg p5 \equiv TerritoryID = 9$$

$$m5 = \neg p2 \wedge \neg p3 \wedge \neg p4 \wedge \neg p5 \equiv TerritoryID \text{ NOT IN } (1..10) \text{ OR NULL}$$

$$\text{Norteamérica} = m1 \vee m2 \quad \text{Europa} = m3 \quad \text{Pacífico} = m4$$

Fragmentos primarios resultantes

Cada minitérmino define un fragmento primario $F_i = \text{minitérmino}(\text{Customer})$:

Frag.	Región / subregión	Predicado (minitérmino)	Clientes
F1	NA1	IN (2,3,4,5)	5,117
F2	NA-2	IN (1,6)	5,311
F3	Europa	IN (7,8,10)	5,727
F4	Pacífico	= 9	3,665
F5	n+1 (red de seguridad)	NOT IN (1..10) OR NULL	

Definición de cada nodo en SQL:

```
-- F1 NA1 : Sur, Centro, Este
SELECT * FROM Sales.Customer WHERE TerritoryID IN (2,3,4,5);

-- F2 NA2 : Noroeste, Canadá
SELECT * FROM Sales.Customer WHERE TerritoryID IN (1,6);

-- F3 Europa
SELECT * FROM Sales.Customer WHERE TerritoryID IN (7,8,10);

-- F4 Pacífico
SELECT * FROM Sales.Customer WHERE TerritoryID = 9;
```

Fragmento n+1 y regla de completitud

La teoría de Özsu exige que ningún dato se pierda, ¿qué pasa si mañana se abre un territorio nuevo (ejem: ***TerritoryID*** = 11) o se inserta un cliente sin territorio (NULL)?

Los 4 nodos anteriores no lo capturarían. El minitérmino m_5 materializa el fragmento n+1, una “red de seguridad” que atrapa cualquier valor futuro o nulo:

```
-- F5 Fragmento n+1 (completitud)
SELECT * FROM Sales.Customer
WHERE TerritoryID NOT IN (1,2,3,4,5,6,7,8,9,10)
  OR TerritoryID IS NULL;
```


Fragmentación derivada

Para que las órdenes y sus dependencias no se separen de su cliente, las tablas miembro se fragmentan por **fragmentación derivada** mediante el operador semi-join ($\bowtie$), que en SQL se traduce a una cláusula *WHERE EXISTS*. La lógica: si un cliente vive en el nodo NA1, todas sus órdenes y los detalles/motivos de esas órdenes heredan esa regla y se van al mismo nodo.

Nivel 1 — *SalesOrderHeader* respecto a cada fragmento F_i de *Customer*:

$$SOH_i = SalesOrderHeader \bowtie (CustomerID) F_i$$

Nivel 2 — *SalesOrderDetail* y *SalesOrderHeaderSalesReason* respecto a SOH_i :

$$SOD_i = SalesOrderDetail \bowtie (SalesOrderID) SOH_i$$

$$SOHSR_i = SalesOrderHeaderSalesReason \bowtie (SalesOrderID) SOH_i$$

Validación de las tres reglas de correctitud

Se verifica el diseño contra los tres criterios de correctitud de la fragmentación:

Completitud

Primaria: como **Pr'** es completo y mínimo, y añadimos el fragmento $n+1$, toda tupla de **Customer** cae en algún fragmento. Ningún registro (actual, futuro o atípico) queda sin nodo.

$$\bigcup_{i=1}^5 F_i = \mathbf{Customer}$$

Derivada: se cumple por integridad referencial. Toda orden tiene un *CustomerID* válido en *Customer*, y todo detalle/motivo tiene un ***SalesOrderID*** válido en ***SalesOrderHeader***; por lo tanto cada tupla miembro encuentra su tupla propietaria en el fragmento correspondiente.

Reconstrucción

La relación global se reconstruye con la unión de sus fragmentos, tanto en la primaria como en la derivada:

$$Customer = F1 \cup F2 \cup F3 \cup F4 \cup F5$$

Disyunción (fragmentos ajenos)

Primaria: los minitérminos son mutuamente excluyentes porque *TerritoryID* toma un único valor por fila, y los conjuntos de la subdivisión de NA (*NA1* y *NA2*) también son disjuntos. Ningún cliente reside en dos nodos.

$$F_i \cap F_j = \emptyset, \forall i \neq j$$

Derivada: el grafo de unión es simple (cada orden pertenece a un solo cliente; cada detalle y cada motivo pertenecen a una sola orden). Al ser una relación de muchos-a-uno hacia el propietario, ninguna tupla miembro se une con propietarios de dos fragmentos distintos, por lo que la disyunción también se garantiza.

Conclusión

El diseño propuesto del ejercicio 3 aplicando de forma completa la metodología de Özsu donde partimos de consultas para así definir las 3 regiones a partir del atributo Group (Norteamérica, Europa, Pacífico) y, al detectar con un análisis de conteo de filas que Norteamérica estaba muy sobrecargada, la subdividimos en dos subregiones balanceadas.

Depuramos el conjunto de predicados con COM_MIN (eliminando el predicado redundante de "toda Norteamérica") para obtener Pr' completo y mínimo y generamos los miniterminos, descartando los contradictorios por la exclusión mutua de *TerritoryID*.

El resultado son 4 nodos de datos un fragmento n+1 que garantiza la completitud ante territorios futuros o valores nulos.

La fragmentación derivada por semi-join propaga la partición del cliente a sus tres tablas dependientes (SalesOrderHeader, SalesOrderDetail y SalesOrderHeaderSalesReason), preservando la localidad: los datos de un cliente y de sus compras viven en un mismo nodo. Finalmente, verificamos que el diseño cumple las tres reglas de correctitud completitud, reconstrucción y disyunción tanto para la fragmentación primaria como para la derivada.